



Informatique graphique avancée, animation et rendu

Compte-rendu TP5

Recalage par ICP

Louis Jean

Master 2 IMAGINE
Université de Montpellier
N° étudiant : 21914083

3 octobre 2024

Table des matières

1	Introduction	2
2	Fonction ICP	2
3	Métriques d'évaluation	3
3.1	Distance RMSD	3
3.2	Distance de Hausdorff	3
4	Recalage de pointsets identiques	4
4.1	Classique	4
4.2	Avec HPSS	6
4.3	Pondérer les sommets	7
5	Recalage de pointsets incomplets	7
6	Recalage de pointsets inconsistents	8
7	Conclusion	8

1 Introduction

Dans ce TP, nous explorons l'algorithme ICP (Iterative Closest Point), utilisé pour le recalage de nuages de points. L'objectif est d'aligner un nuage de points avec un autre nuage de points. Durant tout ce TP, le nuage de points blanc représente le nuage de points source, le nuage de points vert représente le maillage de points à recaler (sur le nuage de points blanc), et le nuage de points rouge représente la transformation calculée par l'algorithme ICP appliquée au nuage de points vert.

2 Fonction ICP

Voici la fonction que j'ai implémentée qui réalise l'ICP. Elle suit l'implémentation classique de cet algorithme.

```
1 void ICP(std::vector<Vec3> const & ps , std::vector<Vec3> const & nps ,
2         std::vector<Vec3> const & qs , std::vector<Vec3> const & nqs ,
3         BasicANNkdTree const & qsKdTree , Mat3 & rotation , Vec3 & translation , unsigned int
4         ↪ nIterations) {
5     Vec3 centroide_ps(0.0f,0.0f,0.0f);
6     Vec3 centroide_qs(0.0f,0.0f,0.0f);
7     for(unsigned int it = 0 ; it < nIterations ; ++it) {
8         for(const auto& p : ps) {
9             centroide_ps += p;
10        }
11        for(const auto& q : qs) {
12            centroide_qs += q;
13        }
14        centroide_ps /= ps.size();
15        centroide_qs /= qs.size();
16        translation = centroide_qs - centroide_ps;
17        std::vector<Vec3> ps_transformed;
18        std::vector<Vec3> qs_transformed;
19        for(auto& p : ps) {
20            ps_transformed.push_back(p - centroide_ps);
21        }
22        for(const auto& q : qs) {
23            qs_transformed.push_back(q - centroide_qs);
24        }
25        Mat3 covariance_matrix = Mat3::Zero();
26        for(const auto& pst : ps_transformed) {
27            unsigned int index = kdtree.nearest(pst);
28            covariance_matrix += Mat3::tensor(qs_transformed[index] , pst);
29        }
30        covariance_matrix.setRotation();
31        rotation = covariance_matrix;
32        std::cout << "\tICP , iteration " << it << " done" << std::endl;
33    }
```

3 Métriques d'évaluation

3.1 Distance RMSD

Pour évaluer les résultats, j'ai utilisé la fonction RMSD (Root Mean Square Deviation), qui calcule la racine de la moyenne des différences point à point de deux nuages de points :

$$\text{RMSD}(P, Q) = \sqrt{\frac{1}{n} \sum_{i=1}^n \|p_i - q_i\|^2}$$

Voici comment je l'ai implémentée.

```
1 double RMSD(const std::vector<Vec3>& points1, const std::vector<Vec3>& points2) {
2     if (points1.size() != points2.size()) {
3         return -1.0f;
4     }
5     double sumSquaredDiffs = 0.0;
6     for (size_t i = 0; i < points1.size(); ++i) {
7         Vec3 diff = points1[i] - points2[i];
8         sumSquaredDiffs += diff.squareLength();
9     }
10    double meanSquaredDiff = sumSquaredDiffs / points1.size();
11    return std::sqrt(meanSquaredDiff);
12 }
```

3.2 Distance de Hausdorff

Pour la partie de ce TP où nous travaillons sur des nuages de points de tailles différentes, la distance RMSD ne fonctionne plus. J'ai donc choisi d'utiliser la distance de Hausdorff :

$$d_H(P, Q) = \max \left\{ \sup_{p \in P} \inf_{q \in Q} \|p - q\|, \sup_{q \in Q} \inf_{p \in P} \|p - q\| \right\}$$

Voici comment je l'ai implémentée.

```
1 double Hausdorff(const std::vector<Vec3>& points1, std::vector<Vec3>& points2) {
2     size_t points1_size = points1.size();
3     size_t points2_size = points2.size();
4     std::vector<double> minDistance(points1_size, std::numeric_limits<double>::max());
5     for (size_t i = 0; i < points1_size; i++) {
6         for (size_t j = 0; j < points2_size; j++) {
7             double distance = Vec3::euclideanDistance(points1[i], points2[j]);
8             if (distance < minDistance[i]) {
9                 minDistance[i] = distance;
10            }
11        }
12    }
13    auto maxMinDistance = std::max_element(std::begin(minDistance), std::end(minDistance));
14    double Hausdorff1 = *maxMinDistance;
15    minDistance.resize(points2_size, std::numeric_limits<double>::max());
16    for (size_t i = 0; i < points2_size; i++) {
```

```

17     for(size_t j = 0; j < points1_size; j++) {
18         double distance = Vec3::euclideanDistance(points2[i],points1[i]);
19         if(distance < minDistance[i]) {
20             minDistance[i] = distance;
21         }
22     }
23 }
24 maxMinDistance = std::max_element(std::begin(minDistance),std::end(minDistance));
25 double Hausdorff2 = *maxMinDistance;
26
27 return std::max(Hausdorff1,Hausdorff2);
28 }

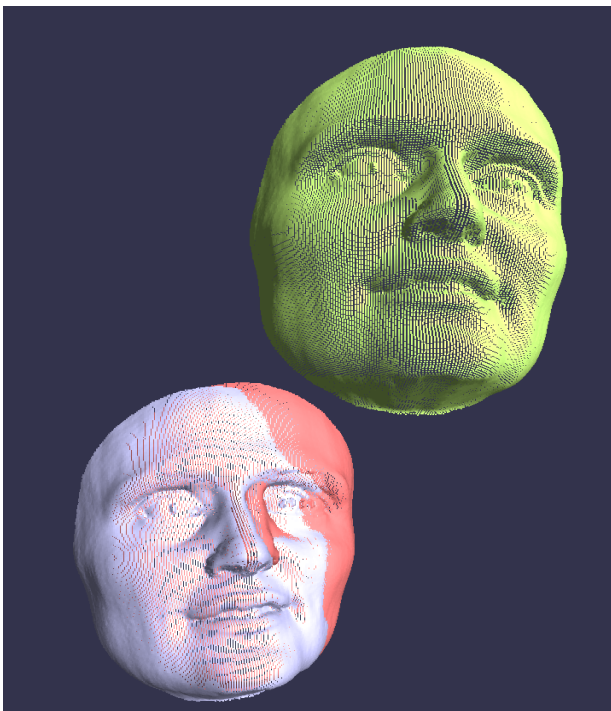
```

4 Recalage de pointsets identiques

Dans cette partie, le nuage de points à recaler est exactement le même que le nuage de points source, mais positionné et tourné aléatoirement dans l'espace.

4.1 Classique

Avec une seule itération, on observe que les résultats changent sensiblement selon les conditions initiales.



(a) Recalage approximatif, RMSD = 0.12



(b) Recalage plus que correct, RMSD = 0.024

Figure 1: Comparaison de deux instances d'ICP avec une seule itération sur *face.pn*

Cependant, en utilisant plus d'itérations, on parvient à obtenir des résultats consistants.

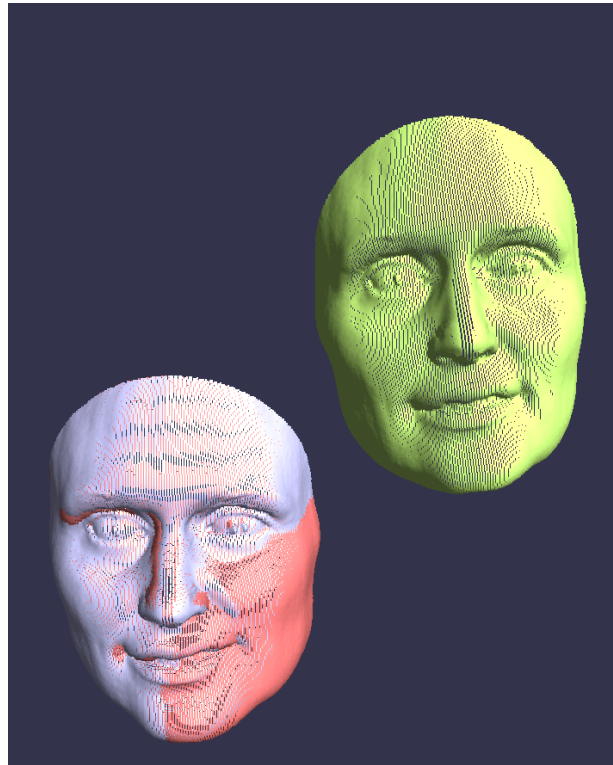
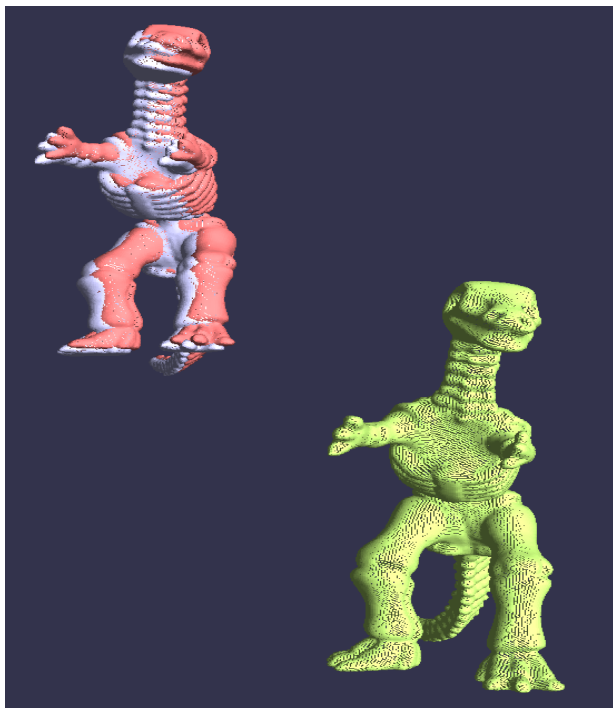
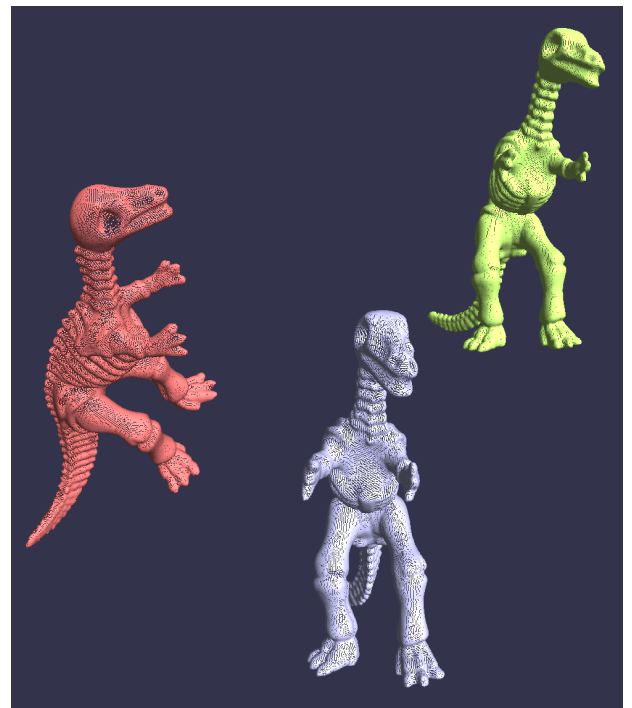


Figure 2: ICP avec 5 itérations sur *face.pn*, $\text{RMSD} = 0.0074$

Pour améliorer les résultats, on peut donc augmenter le nombre d'itérations de l'algorithme.



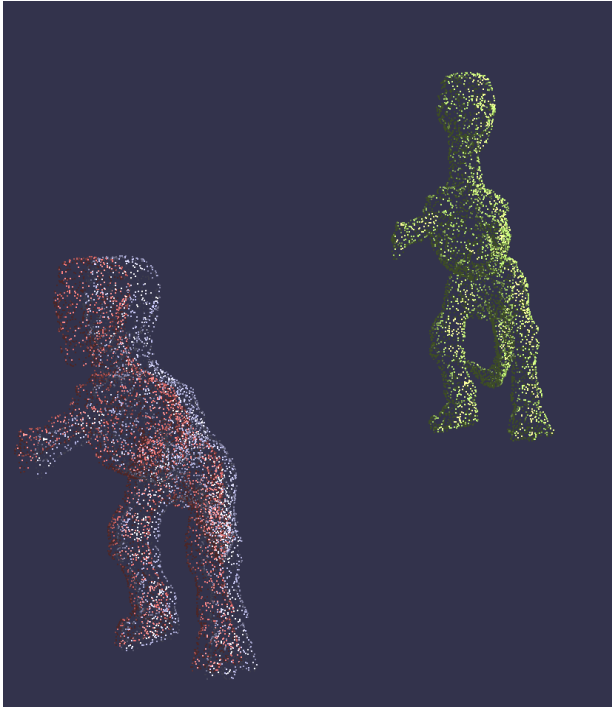
(a) Recalage correct, $\text{RMSD} = 0.033$



(b) Recalage très mauvais, $\text{RMSD} = 1.82$

Figure 3: ICP avec une seule itération sur *dino.pn* et *dino2.pn*

Pour une raison qui m'échappe, *dino2.pn* ne se recalc jamais comme il faut, peu importe le nombre d'itérations. Mon intuition me dit qu'il y a des données "cachées" dans ce nuage de points qui perturbent le fonctionnement de l'ICP.



(a) Recalage correct, $\text{RMSD} = 0.06$



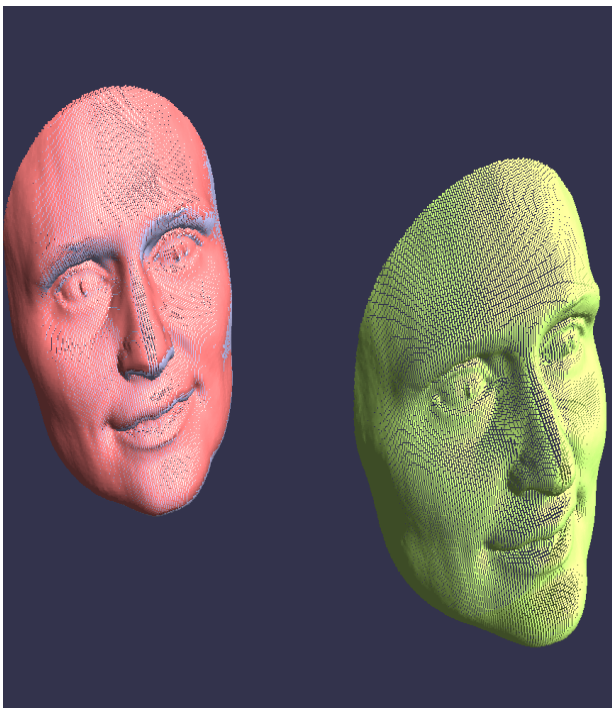
(b) Recalage très mauvais, $\text{RMSD} = 1.25$

Figure 4: ICP avec une seule itération sur *dino_subsampled_extreme.pn* et *dino2_subsampled_extreme.pn*

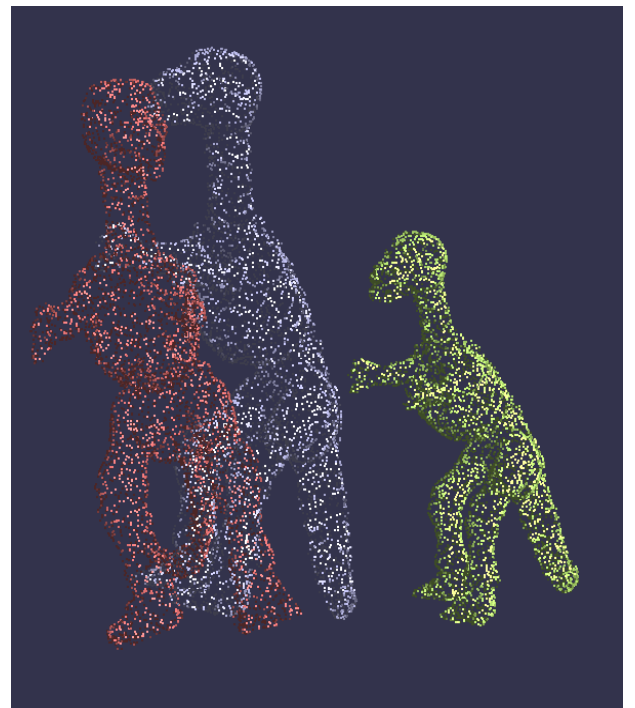
Pour les versions décimées, on retrouve environ les mêmes résultats, toujours pour une seule itération.

4.2 Avec HPSS

En modifiant la fonction ICP pour utiliser une surface HPSS, voici le genre de résultats que l'on obtient pour 5 itérations.



(a) Recalage correct, $\text{RMSD} = 0.014$



(b) Recalage approximatif, $\text{RMSD} = 0.22$

Figure 5: ICP avec 5 itérations et HPSS

Pour le nuage de points *dino.pn*, on remarque une nette amélioration, malgré un recalage toujours approximatif. Cependant, les temps de calculs sont fortement impactés.

4.3 Pondérer les sommets

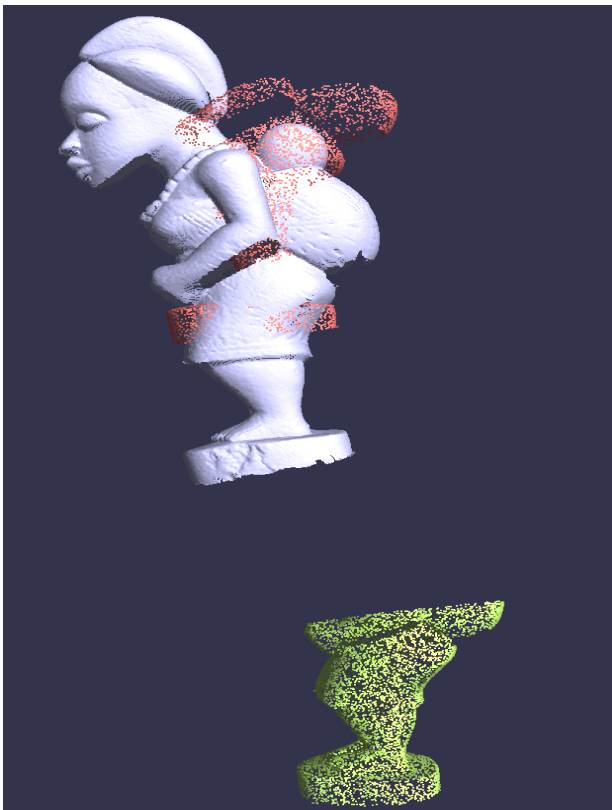
Pour pondérer les sommets avec un poids w_i par sommet p_i , j'ai eu l'idée suivante :

- On initialise tous les poids w_i à 1.
- À chaque itération, on calcule la distance entre le point transformé actuel et le point source.
- À l'itération suivante, on calcule la variation de cette distance.
- Si cette quantité est positive, alors on s'est éloigné de la bonne transformation, donc on réduit le poids associé à ce point. À l'inverse, si cette quantité est négative, alors on s'est rapproché de la bonne transformation, et on incrémente le poids du point.

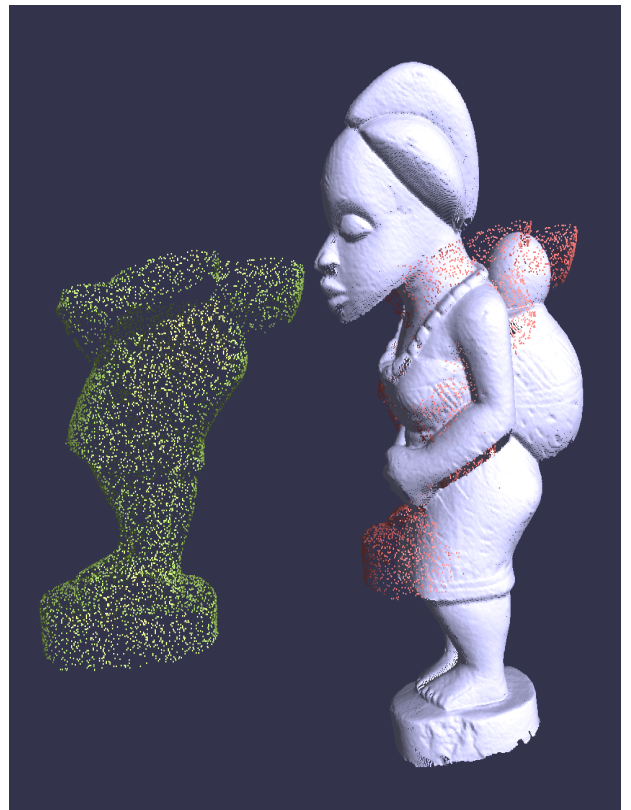
Je ne sais pas si cette méthode est pratique ni efficace, mais intuitivement c'est la première qui m'est venue à l'esprit.

5 Recalage de pointsets incomplets

Dans cette partie, on essaie de recalrer un nuage de points partiellement coupé *african_statue_partial_1.pn* (on entend par là auquel il manque des points par rapport à l'original) sur le nuage de points original *african_statue.pn*.



(a) Sans HPSS, Hausdorff = 0.34



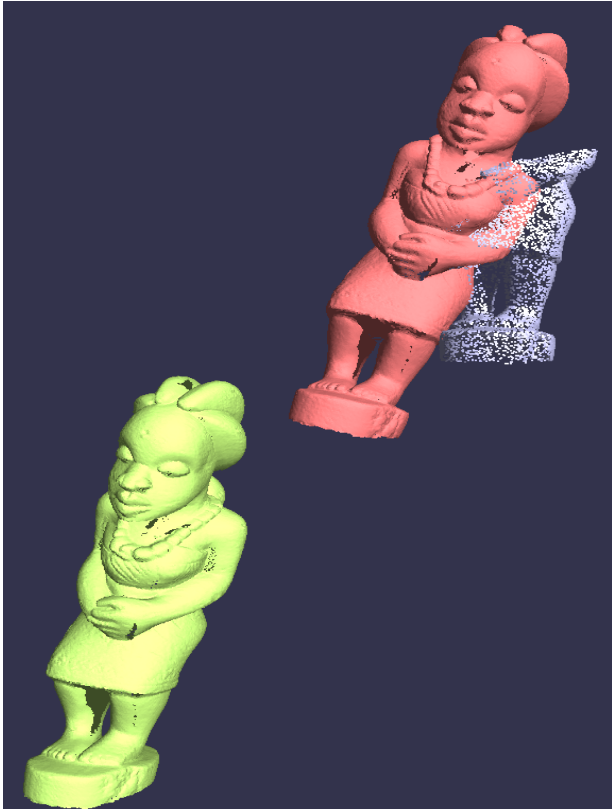
(b) Avec HPSS, Hausdorff = 0.302

Figure 6: ICP avec 5 itérations

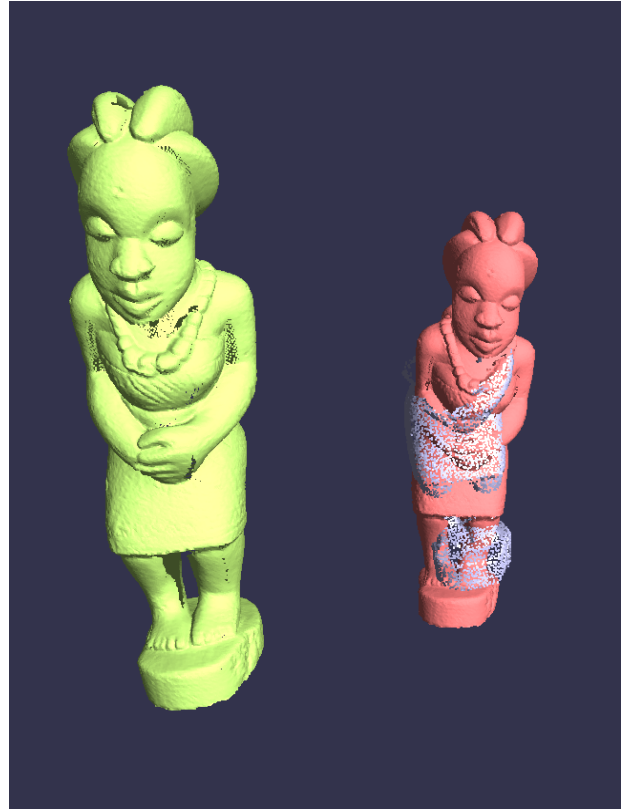
L'ICP montre ici ses limites. Même avec plus d'itérations (j'ai testé jusqu'à 20), je n'obtiens pas de meilleurs résultats.

6 Recalage de pointsets inconsistents

Dans cette dernière partie, on cherche à recaler le nuage de points original *african_statue.pn* sur un nuage de points partiel *african_statue_partial_1.pn*.



(a) Sans HPSS, Hausdorff = 0.46



(b) Avec HPSS, Hausdorff = 0.36

Figure 7: ICP avec 5 itérations

Là encore, l'algorithme a beaucoup de mal à "recoller" les morceaux, et offre des résultats peu satisfaisants.

7 Conclusion

Ce TP nous a permis d'explorer en profondeur l'algorithme ICP pour le recalage de nuages de points, en mettant en place différentes variantes et en testant des scénarios complexes.

L'ICP s'est montré efficace pour des nuages de points identiques ou très proches. Cependant, nous avons constaté que l'algorithme peut être sensible aux conditions initiales, notamment lorsque les nuages de points ne sont pas bien alignés au départ, ou lorsque des parties manquent dans un des ensembles.

Merci pour le temps et l'attention que vous avez consacrés à la lecture de ce compte-rendu, **mes plus plates excuses pour le retard dans le rendu de ce travail.**